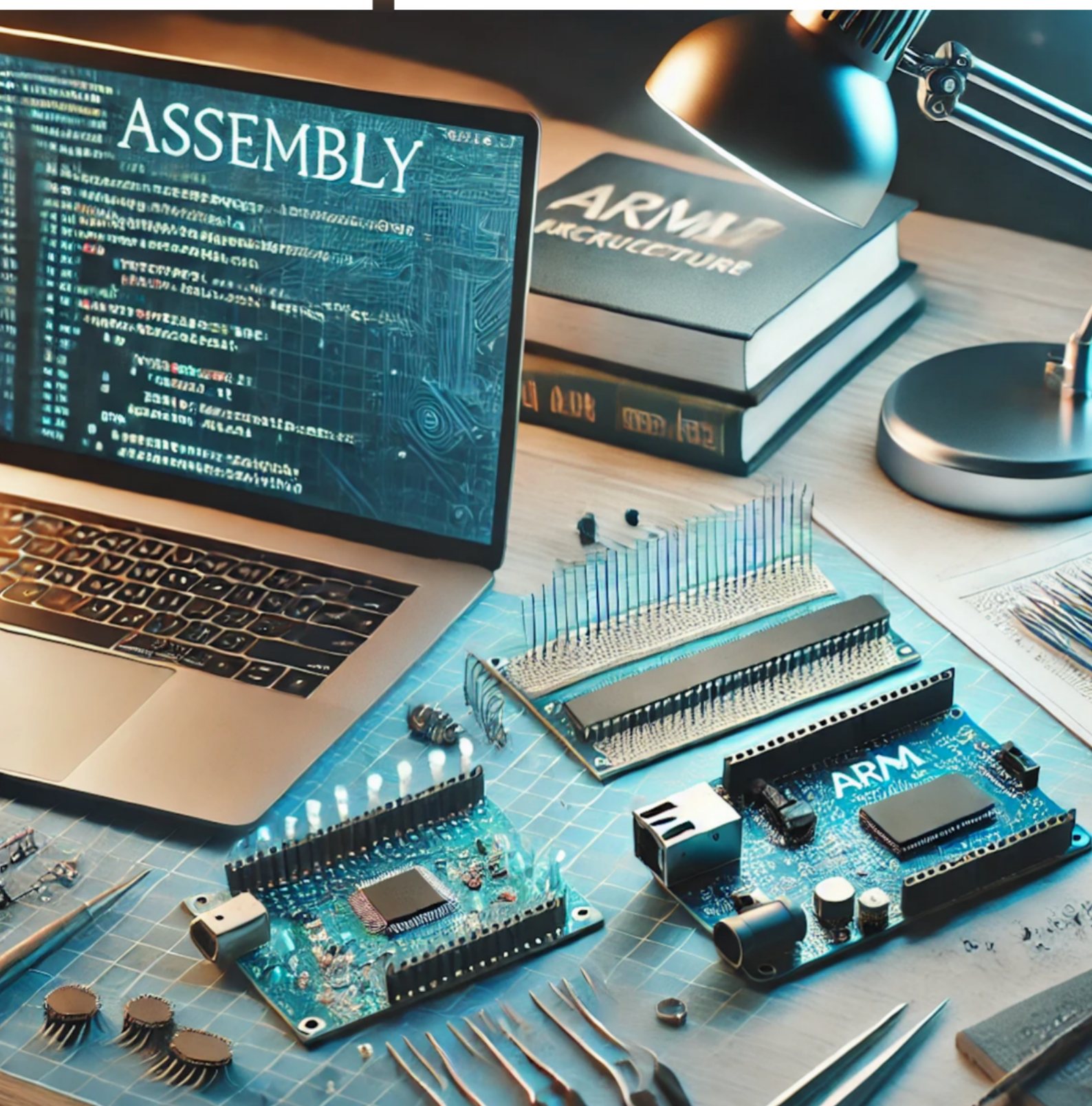


June 2024

微控制器 期末复习讲义

林星辰



前言

初学该学科时，被大量复杂的指令弄得晕头转向，仿佛陷入迷雾之中。碰巧在 YouTube 看到 freeCodeCamp.org 博主发布的大量免费课程，其中 Assembly Language Programming with ARM –Full Tutorial for Beginners 这个视频对 ARM 指令讲解的极为透彻与有趣。它摒弃了由概念填鸭式学习方式，通过实践与引导，一步步记住了不同的 arm 指令，这种引导式学习、解决问题式学习让我尤为兴奋，未曾见过如此学习方式，靠着各种翻译手段，终于弄懂这两个小时的全英文视频。于是本书第三章的编写大量参考了该视频。

本书学习过程中网上的开源资源给了很大帮助，因此，希望本书同样可为知识的开源与共贡献一份力量。

本书正式动笔于考试前，至完成仅用四天，同时身体又有些疾病。如此快速的完成离不开朋友们的帮助与照顾，希望此书可以答谢大家。

时间之仓促，难免有些纰漏，如读者发现，可反馈到 B 站 UP 林鹤鸣 ID (102399253)。写于 2024 年 6 月 20 日 21:28，讲了 6 小时。

ARM 在线模拟地址：<https://cpulator.01xz.net/?sys=arm-de1soc>

版本：0.3

- 修改之前部分错误，如（答案错误、代码错误、STM 示意图）等
- 修改部分代码（冒泡排序、LED 灯），答案更简单
- 第一章基础知识新增
- 修订时间：2024 年 7 月 3 日

目录

1	第一章绪论	3
2	第二章 ARM 嵌入式微处理器	3
2.1	ARM 处理器	3
2.1.1	工作状态	3
2.1.2	工作模式	3
2.2	ARM 寄存器	3
2.2.1	通用寄存器	4
2.2.2	状态寄存器	4
2.3	其他	5
3	第三章 ARM 指令集	7
3.1	概述	7
3.1.1	ARM 指令集机器编码基本格式	7
3.1.2	ARM 汇编语言指令格式	7
3.2	寻址方式	9
3.3	算数指令与 CPSR	9
3.4	移位指令	11
3.5	逻辑运算指令	12
3.6	条件和分支	13
3.6.1	简单分支与条件	13
3.6.2	条件指令	13
3.6.3	条件分支进阶：函数	14
3.7	乘法指令	15
3.8	存储与加载	16
3.8.1	单寄存器存储与加载	16
3.8.2	多寄存器加载与存储指令	18
3.9	立即数合法判断	20
4	第四章 ARM 编程	26
4.1	ARM 伪指令	26
4.2	代码格式	28
5	第五章 C 语言与汇编语言混编	38
5.1	概念	38
5.2	ARM 和 Thumb 混合编程	39
5.3	汇编语言和 C 语言交互编程	41
6	第六章 STM32 输入输出	47
6.1	基本概念	47
6.1.1	GPIO 的 8 种工作模式	47
6.1.2	输入／输出常用固件库函数	47
6.2	编程	47

1 第一章绪论

- 微控制器是把中央处理器、存储器、定时器/计数器、各种输入/输出接口等都集成在一块集成电路芯片上的微型计算机。
- cortex-M3/M4 属于哈佛体系结构
- cortex-M3 采用三级流水线结构，分为：取指、译码、执行
- Cortex-M3 采用 ARMv7 架构
- Arm 公司是成立于 1990 年

2 第二章 ARM 嵌入式微处理器

2.1 ARM 处理器

2.1.1 工作状态

1. ARM 状态：处理器工作于 32 位指令的状态。
2. Thumb 状态 (Thumb-2)：是执行 16 位指令的状态。(Cortex-M 只有 Thumb 状态。)
3. 调试状态：处理器停机调试时进入调试状态。

2.1.2 工作模式

处理器模式	说明	备注
用户 (usr)	正常程序执行模式	不能直接切换到其它模式
系统 (sys)	运行操作系统的特权任务	与用户模式类似，但具有可以直接切换到其它模式等特权
快中断 (fiq)	支持高速数据传输及通道处理	FIQ 异常响应时进入此模式
外部中断 (irq)	用于通用中断处理	IRQ 异常响应时进入此模式
管理 (svc)	操作系统保护模式	系统复位和软件中断响应时进入此模式
中止 (abt)	用于支持虚拟内存或存储器保护	当数据或指令预取终止时进入
未定义 (und)	支持硬件协处理器的软件仿真	未定义指令异常响应时进入此模式

- 特权模式：除用户模式
- 异常模式：除用户与系统模式

2.2 ARM 寄存器

ARM 处理器总共有 37 个寄存器，可以分为以下两类：

2.2.1 通用寄存器

寄存器	描述
R1 ~R15	通用寄存器
R13_svc, R14_svc	服务模式下的栈指针和链接寄存器
R13_abt, R14_abt	中止模式下的栈指针和链接寄存器
R13_und, R14_und	未定义模式下的栈指针和链接寄存器
R13_irq, R14_irq	IRQ 模式下的栈指针和链接寄存器
R8_fiq ~R14_fiq	快速中断模式下的寄存器

- 未分组寄存器
 - R0 ~R7 (8 个)
 - 所有模式共用这 8 个通用寄存器。
- 分组寄存器
 - R8 ~R12 (10 个)
 - * 每个寄存器对应 2 个不同的物理寄存器。
 - * 处理器使用 FIQ 模式时，使用 R8_fiq~R12_fiq (发生 FIQ 中断后，可以加速 FIQ 的处理速度)。
 - * 其他模式下使用 R8_usr~R12_usr。
 - R13 ~R14 (12 个)
 - * 用户模式与系统模式共用 R13_usr, R14_usr。
 - * 5 种异常模式各有各的 R13 和 R14。
 - * R13 常作为堆栈指针 (SP)。
 - * R14 为链接寄存器 (LR)，在结构上有两个特殊功能：
 - (a) 每种模式下，模式自身的 R14 版本用于保存子程序返回地址；
 - (b) 发生异常时，将 R14 对应的异常模式版本设置为异常返回地址 (有些异常有一个小的固定偏移量)。
- 程序计数器 (PC) R15
 - 用于控制程序中指令的执行顺序。正常运行时，PC 指向 CPU 运行的下一条指令。
 - PC 的值可以改变 CPU 的执行流向。

2.2.2 状态寄存器

寄存器	描述
CPSR	当前程序状态寄存器
SPSR_svc	服务模式下的程序状态保存寄存器
SPSR_abt	中止模式下的程序状态保存寄存器
SPSR_und	未定义模式下的程序状态保存寄存器
SPSR_irq	IRQ 模式下的程序状态保存寄存器
SPSR_fiq	快速中断模式下的程序状态保存寄存器

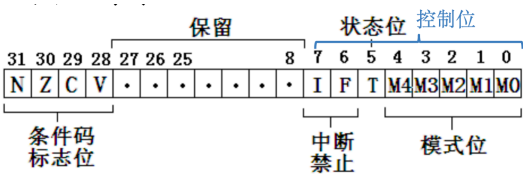


图 1: 当前程序状态寄存器 (current program status register)

条件码	描述	
N (negative)	正负标志；补码表示的符号数运算结果，1 表示负，0 表示正。	
Z (zero)	零标志；1 表示结果为 0，0 表示结果非 0。	
C (carry)	进位标志；加法时 1 表示进位，减法时 0 表示借位，移位操作时为移出值的最后一位。	
V (overflow)	溢出标志；1 表示溢出，0 表示无溢出	
Q	饱和条件码标志位；增强的 DSP 运算指令是否发生了溢出。	
控制位	位	描述
中断禁止位	I	1 表示禁止 (IRQ)，0 表示允许。
	F	1 表示禁止 (FIQ)，0 表示允许。
T 标志位		1 表示 Thumb，0 表示 ARM。
工作模式位	$M_4M_3M_2M_1M_0$	用来标识或设置处理器的工作模式。

表 2: 条件码和控制位描述

2.3 其他

- 32 位的微处理器，ARM 体系支持的最大寻址空间为 4GB (2^{32} 字节)
- 程序正常执行时，每执行一条 ARM 指令，当前 PC 加 4 个字节，每执行一条 Thumb 指令，当前 PC 加 2 个字节。

第二章例题

【例 1】 设 $R0=0x1, R3=0x3$, 指令执行之前 $CPSR=nzcvqIFt_SVC$, 分别执行如下指令 $CPSR$ 的值有何变化?

1) `SUB R1, R0, R3`

2) `SUBS R1, R0, R3`

【例 2】 设 $R0=0x1, R3=0x3$, 指令执行之前 $CPSR=nzcvqIFt_SVC$, 分别执行如下指令 $CPSR$ 的值有何变化?

1) `ADD R1, R0, R3`

2) `ADDS R1, R0, R3`

第二章答案

【答案 1】 1) $R1 \leftarrow R0 - R3$, $CPSR$ 不变

2) $R1 \leftarrow R0 - R3$, 影响标志位 $CPSR=NzcvqIFt_SVC$

【答案 2】 1) $R1 \leftarrow R0 + R3$, $CPSR$ 不变

2) $R1 \leftarrow R0 + R3$, 影响标志位 $CPSR=nzcvqIFt_SVC$

3 第三章 ARM 指令集

3.1 概述

3.1.1 ARM 指令集机器编码基本格式

条件码	指令码	目的寄存器	操作数 1 寄存器	操作数 2
31-28	27-20	19-16	15-12	11-0

- 高 4 位条件位（类比 CPSR）：ARM 指令都是条件执行，先判断条件码与 CPSR 标志位状况是否匹配，相同则执行，否则跳过
- 指令码只有 8 位：精简指令集，最多才 256 个指令
- 目的操作数必须为寄存器：4 位描述 16 个寄存器（R0~R15）
- 操作数 1 也必须为寄存器：故 4 位
 - 例如，即使计算 2+3 也不能是两个立即数，其中一位必定放在寄存器里
- 操作数 2 位数多，可以灵活使用，形式在下文

3.1.2 ARM 汇编语言指令格式

- `<opcode>{<cond>}{S} <Rd>, <Rn>{, <operand2>}`
 - `< >` 内容必不可少
 - `{ }` 中可以省略
 - S 若有 S 后缀，则根据计算结果更新 CPSR 中的条件码；没有则只计算结果
 - `opcode`：操作码（指令助记符）
 - `cond`：可选择条件码。缺省表示使用默认条件 AL（无条件执行）
 - `Rd`：目标寄存器；
 - `Rn`：存放第一个操作数的寄存器；
 - `operand2`：第二个操作数。
 - * `immed_8r`——常数表达式；（立即寻址）
 - * `Rm`——寄存器方式；（寄存器寻址）
 - * `Rm,shift`——寄存器移位方式；（寄存器移位寻址）
- `!` 后缀：指令执行后，修改基址寄存器中的值
 - 基址寄存器中的值（指令执行后）= 指令执行前的值 + 地址偏移量
 - 即 $Rn = Rn + Op2$

编码	后缀助记符	标志位	定义
0000	EQ	$Z = 1$	(相等) 零标志: 结果为零时为真
0001	NE	$Z = 0$	(不相等) 非零标志: 结果不为零时为真
0010	CS/HS	$C = 1$	(无符号数大于或等于) 进位标志
0011	CC/LO	$C = 0$	(无符号数小于) 未进位标志
0100	MI	$N = 1$	(负数) 负标志: 结果为负时为真
0101	PL	$N = 0$	(正数或零) 非负标志: 结果为非负时为真
0110	VS	$V = 1$	溢出
0111	VC	$V = 0$	未溢出
1000	HI	$C = 1$ 且 $Z = 0$	无符号大于
1001	LS	$C = 0$ 或 $Z = 1$	无符号小于等于
1010	GE	$N = V$ (N 和 V 相同)	大于等于 (有符号数)
1011	LT	$N \neq V$ (N 和 V 不同)	小于 (有符号数)
1100	GT	$Z = 0$ 且 $N = V$	大于 (有符号数)
1101	LE	$Z = 1$ 且 $N \neq V$	小于等于 (有符号数)
1110	AL	无	总是 (Always): 无条件执行
1111	NV	无	从不执行 (不要使用)

表 3: 条件码编码及定义

寄存器类别	寄存器在汇编中的名称	各模式下实际访问的寄存器						
		用户	系统	管理	中止	未定义	中断	快中断
通用寄存器和程序计数器	R0(a1)	R0						
	R1(a2)	R1						
	R2(a3)	R2						
	R3(a4)	R3						
	R4(v1)	R4						
	R5(v2)	R5						
	R6(v3)	R6						
	R7(v4)	R7						
	R8(v5)	R8						R8_fiq
	R9(SB,v6)	R9						R9_fiq
	R10(SL,v7)	R10						R10_fiq
	R11(FP,v8)	R11						R11_fiq
	R12(IP)	R12						R12_fiq
	R13(SP)	R13		R13_svc	R13_abt	R13_und	R13_irq	R13_fiq
	R14(LR)	R14		R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
状态寄存器	R15(PC)	R15						
	CPSR	CPSR						
	SPSR	无		SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq

表 4: 各模式下实际访问的寄存器

3.2 寻址方式

```

1      ; 立即寻址
2      MOV R0,#0xFF00    ; R0 ← 立即数0xFF000
3      SUB R0,R0,#5      ; R0 ← R0 - 5
4      ; 寄存器寻址
5      mov r1, r2        ; R1 ← R2
6      SUB r0, r1, r2    ; R0 ← R1 - R2
7      ; 寄存器间接寻址
8      ; load data from stuck into registers
9      LDR R0, [R1]      ; R0 ← [R1]
10     SWP R1,R1,[R2]    ; 将R1和[R2]的内容交换
11     ; 基址变址寻址
12     LDR R0, [R1, #4]  ; R0 ← [R1 + 4]
13     ; 前索引
14     LDR R0, [R1, #4]! ; R1 ← R1 + 4 , R0 ← [R1]
15     ; 后索引
16     LDR R0, [R1], #4  ; R0 ← [R1] , R1 ← R1 + 4
17     LDR R0, [R1, R2]  ; R0 ← [R1 + R2]

```

Listing 1: ARM 寻址方式

```

1  int list[7] = {4, 5, -9, 1, 0, 2, -3};
2  int R1 = 0;
3  R0 = list[R1];      // LDR R0, [R1]
4  R0 = list[R1+1];    // LDR R0, [R1, #4]
5
6  R1 = R1 + 1;
7  R0 = list[R1];      // LDR R0, [R1, #4]!
8
9  R0 = list[R1];      // LDR R2, [R0], #4
10 R1 = R1 + 1;

```

Listing 2: * 读取数据方式与 C 语言比较

3.3 算数指令与 CPSR

- add 加 subtract 减 multiply 乘
- N negative 负数、Z zero 结果为零、C carry in 进位、V overflow 溢出
- why: 只用如此低层次语言就是为了提高效率, 节约资源, 设置 CPSR 会增加额外操作
- when 使用 subs: 确定结果是负数或不知道减法结果 (使用加载的数减时)

```

1      ; 初识
2      MOV R0, #5
3      MOV R1, #7
4      ADD R2, R0, R1 ; R2 = R0 + R1
5      SUB R2, R0, R1 ; R2 = R0 - R1
6      ; 问题
7      MOV R0, #0xFFFFFFFF
8      MOV R1, #1
9      SUB R2, R0, R1
10     ; 初识 CPSR
11     MOV R0, #5
12     MOV R1, #7
13     SUBS R2, R0, R1
14     ; 进位及带进位加法
15     MOV R0, #0xFFFFFFFF
16     MOV R1, #3
17     ADDS R2, R0, R1
18     ADC R2, R0, R1 ; R2 = R0 + R1 + carry

```

Listing 3: * 初识算数指令

小提示：进位

例如计算 $39 + 14$ ，假设这种寄存器只能存储一个数字，即 $9 + 4 = 3$ ，进位标志位 C 激活为 1， $3 + 1 + C = 5$ ，故答案是 53。

```

1      ; ADD 加法指令
2      ADD R0, R1, R2 ; R0 = R1 + R2
3      ADD R0, R1, #5 ; R0 = R1 + 5
4      ADDS R0, R1, R2, LSL #2 ; R0 = R1 + ( R2 * 4 )
5      ; ADC 带进位加法指令
6      ADC R0, R1, R2 ; R0 = R1 + R2 + C
7      ADC R0, R1, R2, LSL #2 ; R0 = R1 + ( R2 * 4 ) + C
8      ; SUB 减法指令
9      SUB{S} Rd, Rn, operand2 ; Rd=Rn- operand2
10     ; SBC 带借位减法指令
11     SBC{S} Rd, Rn, operand2 ; Rd=Rn-operand2-C
12     ; RSB 逆向减法指令
13     RSB R0, R1, R2 ; R0 = R2 - R1
14     RSB R0, R1, #6 ; R0 = 6 - R1
15     RSB R0, R2, R3, LSL #1 ; R0 = (R3 * 2) - R2

```

Listing 4: 算数指令

3.4 移位指令

接下来，我们从位级别来操作数字

- LSL: 逻辑左移 (Logical Shift Left)
 - 寄存器中字的低端空出的位补 0。(移动的最后一位放入 C 标志位)
 - 如: 00001010 逻辑左移一位变成 00010100
 - 由 10 变为 20, 可作为一种快速的乘 2 的方法
- LSR: 逻辑右移 (Logical Shift Right)
 - 寄存器中字的高端空出的位补 0。(移动的最后一位放入 C 标志位)
 - 如: 00001010 逻辑右移一位变成 00000101
 - 由 10 变为 5, 可作为一种快速的除 2 的方法
- ROR: 循环右移 (Rotate Right)
 - 逻辑右移时被丢弃的位会循环到左端
 - 如: 00000101 循环右移一位变成 10000010
 - 主要用于哈希、加密和图形处理等领域, 在实际编程中并不常见, 但了解其存在和用途仍然很重要。
 - ARM 中没有 ROL, 但是可以使用 ROR 32-n 位 (希望向左移动的位数实现)
- ASR: 算术右移 (Arithmetic Shift Right)
 - 算数移位的对象是带符号数, 在移位过程中必须保持操作数的符号不变。若正数, 空位补 0。若负数, 补 1
 - 如: 101 ... 101 算术右移变成 1101 ... 10
- RRX: 带扩展位的循环右移 (Rotate Right eXtended by 1 place)
 - 操作数右移 1 位, C 标志位移入最高位, 最低位移入 C 位。
 - 如: 101 ... 011 (C 位为 0) 扩展循环右移之后为 0101 ... 01 (C 位为 1)

```

1  MOV R0, #10
2  LSL R0, #1 ; #后面跟着左移位数, 也即乘2的幂次
3  LSR R0, #1
4
5  MOV R0, #10
6  MOV R1, R0
7  LSL R1, #1
8  ; 上两句等价于下面这一句
9  MOV R1, R0, LSL #1
10
11 MOV R0, #15
12 ROR R0, #1

```

Listing 5: * 验证移位指令

3.5 逻辑运算指令

- 与 (AND)、或 (OR)、异或 (Exclusive OR) 和非 (Negation)
- 注意不是 OR 而是带两个 R 的 OR
- 这些是非常基本的逻辑运算，它们非常简单直接，在各种计算机基础课程中都见过（计硬、计网、OS）没有什么特别之处，稍微不同的是非
- MVN (Move Negative) 将源的取反值移动到目标位置，即取反源值，然后将结果放入目标位置：它同时执行移动和取反操作。
- 取反操作对整个寄存器进行，而不仅仅是单个位。
- ORR 常用于置 1 操作
- AND 常用于清零的操作
- EOR 常用于反转操作数 1 的某些位。（相同为 0，不同为 1）

```
1  MOV R0,#0xFF
2  MOV R1,#22
3
4  AND R2,R0,R1
5  ORR R2,R0,R1
6  EOR R2,R0,R1
7
8  MOV R0,#0xAA
9  MVN R0,R0
10 AND R0,#0x000000FF
11
12 ; BIC 位清除指令
13 ; 汇编格式: BIC {S} Rd, Rn, operand2
14 ; 功能: Rd=Rn AND (!operand2)
15 BIC R0, R0 ,#0x1F ; 将R0最低5位清零，其余位不变。
```

Listing 6: * 探究逻辑运算指令

3.6 条件和分支

3.6.1 简单分支与条件

- CMP 比较指令: compare
- B: branch
- GT: grater than
- 分支指令允许我们根据比较的结果在程序中不同的位置之间移动，与高级语言语言中 if 语句的工作方式非常相似。

```

1      ; 验证跳转
2      MOV R0, #1
3      MOV R1, #2
4      CMP R0, R1
5      BGT greater
6      MOV R2, #2
7  greater:
8      MOV R2, #1
9      ; 将 R0 改为 #3，测试是否跳转，观察 CPSR
10     ; 发现将6行跳过；改为 BAL default，并在结尾增加如下两条语句
11  default:
12     MOV R2, #2

```

Listing 7: * 探究条件与分支

```

1  r0 = 1;
2  r1 = 2;
3  if( r0 > r1 ){
4      r2 = 1;
5  }
6  r2 = 2;

```

Listing 8: * 条件分支与 C 语言比较

3.6.2 条件指令

- 条件指令允许我们根据一个条件来完成特定的指令，类似于分支中比较
- 先根据 CPSR 判断条件后缀是否满足，若满足则执行前面指令
- 将代码 7 的分支用条件指令改写
- 因为一直运行分支是非常复杂的，我们不想每次都这样做。所以这允许我们减少应用程序的复杂性，总体上写出更简单的代码。

```

1  MOV R0, #3
2  MOV R1, #2
3  CMP R0, R1
4  ADDGT R2, #1
5  MOV R2, #2
6  ; 将 GT 换成 LT 观察结果
7  ; 互换 CMP 将CMP的 R0, R1 互换位置, 观察结果

```

Listing 9: * 探究条件指令

特例研究: ADDEQS R1,R0,R2

- 功能: 指令在 Z=1 时执行, $R1 \leftarrow R0 + R2$, 并刷新条件标志位
- 要点: 两种后缀同时存在时, 书写时 S 排在后面
- 条件后缀要测试的是执行前的标志位,
- S 后缀是依据指令执行后的结果改变条件标志。

3.6.3 条件分支进阶: 函数

- 初识函数: 使用链接寄存器进行分支并返回
- 与之前相同写一个可以加两个数字的函数, 在 BAL 后增加一条指令, 试想如何才能执行这条指令。
- 当然可以使用跳转, 如代码 10, 但太麻烦
- 当我们只是将其作为一个函数调用时, 我们可以加入一个称为返回地址的概念。通过返回地址, 我们可以在这个函数结束时返回到我们开始的地方。
- 方法: 改为 BL(branch links) 分支链接, 可以将分支的地址存储在一个寄存器中, 即 LR(links register)

```

1  MOV R0, #2
2  MOV R1, #4
3  BAL add2
4  back:
5  MOV R3, #5
6
7  add2:
8  ADD R2, R0, R1
9  BAL back

```

Listing 10: * 探究函数之跳转


```
1  MOV R0, #2
2  MOV R1, #4
3  BL add2
4  MOV R3, #5
5
6  add2:
7      ADD R2, R0, R1
8      bx lr
```

Listing 11: * 探究函数之链接寄存器

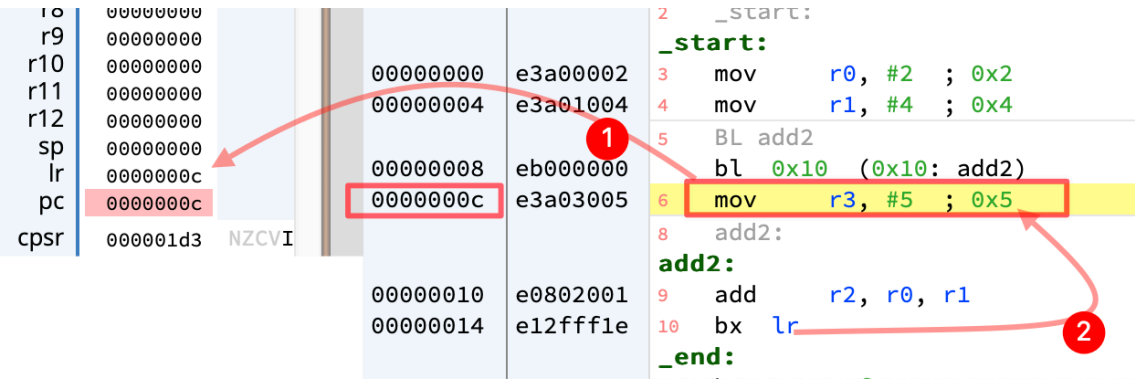


图 2: bl 跳转示意图

3.7 乘法指令

- MUL: 32 位乘法指令
- MLA: 32 位乘加指令
- SMULL: 64 位有符号数乘法指令
- SMLAL: 64 位有符号数乘加指令
- *UMULL: 64 位无符号数乘法指令
- *UMLAL: 64 位无符号数乘加指令

小提示：乘法指令注意事项

- 64 位乘法结果存放在 2 个 32 位的寄存器 Rdlo 和 Rdhi 中。Rdlo 存放低 32 位，Rdhi 存放高 32 位。
- 所有源操作数、目的寄存器必须为通用寄存器（R15 不可用）
- 不能对操作数使用立即数或被移位的寄存器
- 目的寄存器 Rd（包括 Rdlo、Rdhi）和操作数 Rm 必须是不同的寄存器

```

1      ; MUL 32位乘法指令
2      MUL   R0, R1, R2      ; R0 = R1 × R2
3      MULS  R0, R1, R2      ; R0 = R1 × R2, 同时设置CPSR
4      ; MLA 32位乘加指令
5      ; 格式: MLA {S} Rd,Rm,Rs,Rn
6      MLA   R0, R1, R2, R3   ; R0 = R1× R2+ R3
7      MLAS  R0, R1, R2, R3   ; R0 = R1× R2+ R3, 同时设置CPSR
8      ; SMULL 64位有符号数乘法指令
9      SMULL  R0, R1, R2, R3
10             ; R0 = (R2×R3)的低32位 Rdlo
11             ; R1 = (R2×R3)的高32位 Rdhi
12      ; SMLAL 64位有符号数乘加指令
13      SMLAL  R0, R1, R2, R3
14             ; R0 = (R2×R3)的低32位 + R0 Rdlo
15             ; R1 = (R2×R3)的高32位 + R1 Rdhi

```

Listing 12: 乘法指令

- SMLAL 64 位有符号数乘加指令
- 格式: SMLALS Rdlo,Rdhi,Rm,Rs
- 功能: [Rdhi Rdlo]= [Rdhi Rdlo]+ Rm×Rs
- 目的寄存器 Rdlo, 指令执行前存放 64 位加数的低 32 位, 指令执行后存放结果的低 32 位
- 目的寄存器 Rdhi, 指令执行前存放 64 位加数的高 32 位, 指令执行后存放结果的高 32 位

3.8 存储与加载

3.8.1 单寄存器存储与加载

- 如果有 ±, 代表基准寄存器 Rn ± 立即数/寄存器/地址偏移量
- LDR/STR 字数据加载/存储指令

```

1      ; 使用标号
2      LDR R4,START
3      STR R5,DATA1
4      ; 前索引
5      LDR R0, [R1]
6      LDR R0, [R1, R2]
7      LDR R0, [R1, # 8]
8      LDR R0, [R1, R2, LSL # 2]

```

```

9      ; 自动索引
10     STR R0, [R1, R2]!
11     STR R0, [R1, #8]!
12     STR R0, [R1, R2, LSL #2]!
13     ; 后索引
14     LDR R0, [R1], #8
15     LDR R0, [R1], R2
16     LDR R0, [R1], R2, LSL #2

```

Listing 13: 单寄存器加载与存储指令

小提示：加载存储指令注意事项

- 立即数绝对值不大于 4095 的数值
- 偏移量必须保证偏移结果能够使地址对齐
- 不能对操作数使用立即数或被移位的寄存器
- 移位的位数不能超过规定的数值
- R15 作为基址寄存器 Rn 时，不可使用“!”后缀，“!”有回写功能
- R15 不可作为偏移寄存器使用
- LDRH 指令将一个 16 位的半字数据加载到 Rd 中，同时将寄存器的高 16 位清零。且半字状态立即数偏移使用的是 8 位无符号数，

```

1      ; LDRB/STRB 字节数据加载/存储指令
2      LDRB R0, [R1]
3      LDRB R0, [R1, #8]
4      STRB R0, [R1]
5      STRB R0, [R1, #8]
6      ; LDRD/STRD 双字加载/存储指令
7      LDRD R3, R4, [R0], #8      ; R3=[R0], R4=[R0+4], R0=R0+8
8      STRD R3,R4,[R0]            ; R3 → [R0], R4 → [R0+4]
9      ; 说明以下功能，判断对错
10     LDRH R0, [R1]
11     LDRH R0, [R1, #8]
12     LDRH R0, [R1, R2]
13     STRH R0, [R1]
14     STRH R0, [R15, #-0x20]!      ; R15 作为基址不可写回
15     LDRH R0, [R1, #256]          ; 偏移量超出 8 位
16     STRH R0, [R1, R2, LSL #2]    ; 不可使用寄存器移位寻址

```

Listing 14: 单寄存器加载与存储指令

3.8.2 多寄存器加载与存储指令

- STM:(store much) 多数据存储, 将寄存器的值存到地址上
- LDM:(load much) 多数据加载, 将地址上的值加载到寄存器上
- 多寄存器用“{ }”包含, 连续寄存器使用“-”间隔, 否则用“,”分隔;
- 基址寄存器变化方式:
 - IA (Increment after Operating): 操作完后地址递增
 - IB (Increment before Operating) : 地址先增后完成操作
 - DA (Decrement after Operating) : 操作完后地址递减
 - DB (Decrement before Operating) : 地址先减后完成操作

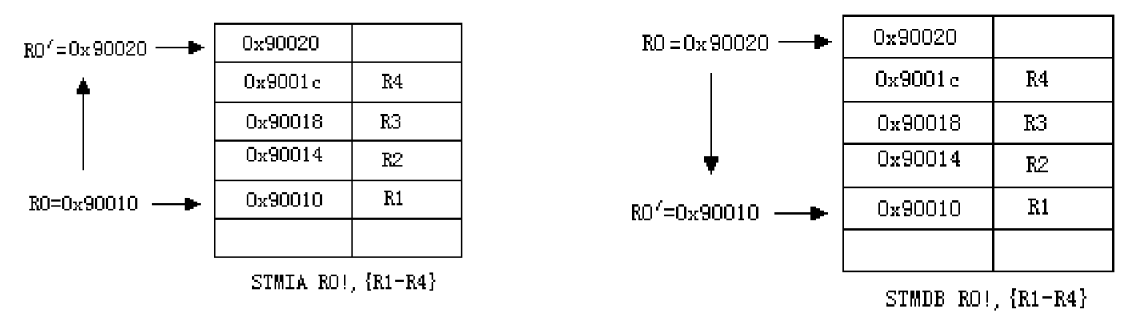


图 3: 基址寄存器变化示意图

- 堆栈组织生长方式
 - FD/ED: 满递减/空递减; (Full Descending / Empty Descending)
 - FA/EA: 满递增/空递增; (Full Ascending / Empty Ascending)
 - 空递增: 堆栈向上增长, SP 指向堆栈上的第一个空位置。指令为 STMEA;
 - 满递减: 堆栈向下增长, SP 指向内含有效数据项的最低地址。指令为 LDMFD、STMFD;

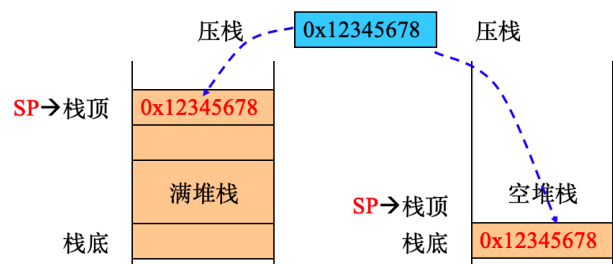


图 4: 空满堆栈示意图

Register	Value	Address	Memory contents and ASCII
R0	00000001	ffffff10	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
R1	00000003	ffffff20	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
R2	00000004	ffffff30	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
R3	00000000	ffffff40	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
R4	00000000	ffffff50	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
R5	00000000	ffffff60	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
R6	00000000	ffffff70	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
R7	00000000	ffffff80	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
R8	00000000	ffffff90	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
R9	00000000	ffffffa0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
R10	00000000	ffffffb0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
R11	00000000	ffffffc0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
R12	00000000	ffffffd0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
SP	ffffff4	ffffffe0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
LR	00000000	fffffff0	00000001 00000003 00000004
PC	00000010	00000000	e3a00001 e3a01003 e3a02004 e92d0007
CPSR	000001d3 NZCVI SVC	00000010	eb000000 e8bd0007 e3a00005 e3a01007
SPSR	00000000 NZCVI ?	00000020	e0802001 e12fff1e aaaaaaaaaa aaaaaaaaaa
S0	00000000		
S1	00000000		
S2	00000000		

图 5: push_pop 示意图

```

1 .global _start
2 _start:
3     MOV R0, #1
4     MOV R1, #3
5     MOV R2, #4
6     PUSH {R0, R1, R2}
7     BL get_value
8     POP {R0 - R2}
9 get_value:
10    mov r0, #5
11    mov r1, #7
12    add r2, r0, r1
13    BX LR

```

Listing 15: * 探究堆栈存储

小提示：为什么进行压栈出栈操作

在 ARM 中我们拥有的寄存器数量是有限的，当需要在函数内部重用已经在函数外部使用的寄存器时，我们需要在调用函数之前保存原本的值，并在函数完成后恢复它们。类似于高级语言中局部变量概念。

- LDMFD 指令用来支持 POP 操作（出栈）
- STMFD 指令用来支持 PUSH 操作（进栈）
- 堆栈指针 SP（R13）：指示现在堆栈中存放数据的位置。
- 一个 32 位数据进栈时，SP（R13 - 4），向下浮动指向下一个地址，即新的栈顶。
- 当数据出栈时，SP（R13+ 4）向上浮动，指向新的栈顶。

- 注意数据进出栈顺序
- STMFD 指令相当于 STMDB 指令，LDMFD 相当于 LDMIA 指令

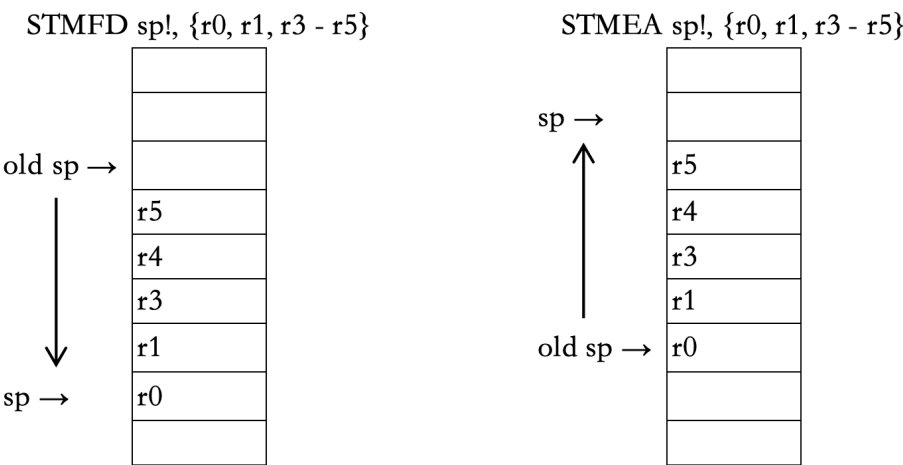


图 6: 入栈出栈示意图

3.9 立即数合法判断

通过观察 ARM 编码格式，发现操作数只有 12 位。12bits 可以表示的无符号数范围为：0 ~ 4095，有符号数的范围为：-2048 ~ 2047，因此我们需要其他操作来表达更多数字。

因此，ARM 规定：立即数是由一个 8 位的常数循环右移偶数位得到的，其中循环右移的位数由一个 4 位 2 进制的两倍表示。

为了扩展得到 32 位，用 12 位中的 8 位表示数据值，剩下 4 位表示移位值。那么这 8 位数字最多可以向左移动 15 位，每移动移位，原数字乘 2，可以表示更多数据。

但这最多才能表示 23 位的数字，还是不够 32 位。于是利用 ALU 的内部结构设计将 4 位移位值乘以 2，即最多可移动 30 位，这样就能用 12 位表示 32 位常数了。

通过上面叙述，会发现立即数都是由 8 位基本数字移位得到，但由于移位只是乘以 2 的次方且只移位偶次，因此有些数字是无法取到的，判断哪些数字合法哪些数字非法可利用以下方式。

- 最高位 1 至最低位 1 之间的二进制数序列的位数不超过 8 位，不足 8 位在高位补 0
- 立即数的二进制序列的右边为偶数个连续的 0

举几个例子

- 0x4FF，不是立即数，0100 1111 1111 大于 8 个
- 0x122，不是立即数，0001 0010 0010 奇数个 0
- 0xF000000F，是立即数，1111 0000 0000 0000 0000 0000 1111，循环右移四位
- 0x8000007F，不是立即数

第三章例题

【例 1】已知 R0=0x00000000, R1=0x80000004, 执行指令后 R0, R1 的值有何变化?

MOV R0, R1, LSL#1

【例 2】已知 R0=0x00000000, R1=0x80000004, CPSR=nzcvqiFt_USER, 执行指令后 R0, R1 的值有何变化?

MOVS R0, R1, LSL#1

【例 3】已知 R0=0xFFFFFFFF, R1=0x00000001, CPSR=nzcvqiFt_USER, 执行指令后 R0, R1 的值有何变化?

MOVS R0, R1, LSR#1

【例 4】已知 R0=0x00000000, R1=0x80000001, CPSR=nzcvqiFt_USER, 执行指令后 R0, R1 的值有何变化?

MOVS R0, R1, ASR#1

【例 5】已知 R0=0x00000000, R1=0x40000001, 执行指令后 R0, R1 的值有何变化?

MOV R0, R1, ROR#1

【例 6】已知 CPSR=nzcvqiFt_USER, R0=0x00000000, R1=0x80000001, 执行指令后 R0, R1, CPSR 中的值有何变化?

MOVS R0, R1, RRX

【例 7】用 ADC 指令完成 64 位加法 (减法), 设第一个 64 位操作数放在 R3、R2 中, 第二个 64 位操作数放在 R5、R4 中。64 位结果放在 R1、R0 中。

【例 8】同上题, 但使用 SBC 实现 64 位减法, 结果存于 R1、R0 中。

【例 9】比较两个值的大小, 并进行加减运算, 以下是 C 代码, 请用条件指令改写。

```
1 if( a < b ) a = a + b;
2 else      a = a - b;
```

【例 10】设 R0 = 0xFFFFFFFF, 则执行下列指令后 R0 的值是多少?

AND R0, R0, # 0xF

【例 11】若要将 R0 的第 0 位和第 3 位设置为 1, 其余位不变则应执行什么指令?

【例 12】若要将 R0 的低 4 位取反, 其余位不变则应执行什么指令?

【例 13】下列乘法指令是否错误? 若错误, 请指出错误的原因。

```
1 MUL R1, R1, R6
2 MUL R1, R6, [R5]
3 MLA R4, R5, R6
4 MULSNE R6, R3, R0
5 UMLAL R6, R4, R4, R1
6 SMULL R5, R3, R0
7 MUL R2, R5, #0x20
```

【例 14】 下列存储指令是否错误？若错误，请指出错误的原因

```

1      LDR R1,[R2,R5]!
2      STR R2,[R3],#0xFFFF8
3      STREQ R4,[R0,R4,LSL R5]
4      STREQ R3,[R6],#-0x08
5      LDR R0,[R2]!,-R6
6      LDR R4,START
7      LDR R1,[SP,#-0x04]
8      LDR PC,R6
9      LDR PC,[R6]
10     LDR R1,[R3,R15]

```

【例 15】 LDR 指令举例，假设执行前

```

1      Mem32[0x90000]=0x00000001
2      Mem32[0x90004]=0x00000002
3      Mem32[0x90008]=0x00000003
4      R0=0x00000000
5      R1=0x00090000
6      R4=0x00000004

```

LDR R0,[R1,R4]，则执行后 R0、R1、R4 有何变化？

LDR R0,[R1,#0x4]，则执行后 R0、R1、R4 有何变化？

LDR R0,[R1],#0x4，则执行后 R0、R1、R4 有何变化？

【例 16】 假设执行前各寄存器的值 R1=0x00000001、R2=0x00000002、R3=0x00000003、R4=0x9000、R5=0x9010

(存储器图由上 0x9014 至下 0x9000 共 6 块)

STMIA R4!, R1,R2,R3，各寄存器和存储器变化？

STMDB R5!, R1,R2,R3，各寄存器和存储器变化？

【例 17】 指令执行之前 SP = 0x00090010(R13), R4=0x00000003, R3=0x00000002, R2=0x00000001, 将 R2-R4 入栈，然后出栈，注意进栈和出栈后 SP 的值？

STMFD SP!,R2-R4

LDMFD SP!,R5-R7

【例 18】 下面那个立即数是不合法的 ()

A.0xFF B.0x3F0 C.0x314 D.0x101

第三章答案

【答案 1】 执行指令后 R0=0x00000008, R1=0x80000004

【答案 2】 执行指令后 R0=0x00000008, R1=0x80000004, CPSR=nzCvqiFt_USER
R1 左移 1 位时最高位的 1 被移出, 这会影响 C 标志位。注意: 溢出标志位 (V) 主要在执行加法和减法运算时会受到影响。

【答案 3】 执行指令后 R0=0x00000000, R1=0x00000001, CPSR=nZCvqiFt_USER
验证移位时如何影响 C、N 标志位, 更改移位位数及 LSR/LSL。

```
1      mov    r1, #0xf0000003
2      movs   r0, r1, LSR #2
```

【答案 4】 执行指令后 R0=0xC0000000, CPSR=NzCvqiFt_USER

【答案 5】 执行指令后 R0=0xA0000000, R1=0x40000001

【答案 6】 执行指令后 R0=0x40000000, CPSR=nzCvqiFt_USER,

【答案 7】 答案如下

```
1      ADDS   R0,R2,R4 ;R0 等于低 32 位相加, 并影响标志位
2      ADC    R1,R3,R5 ;R1 等于高 32 位相加, 并加上低位进位
```

【答案 8】 答案如下

```
1      SUBS   R0,R2,R4 ; 低 32 位相减, 并影响标志位
2      SBC    R1,R3,R5 ; 高 32 位相减, 并减去低位借位
```

【答案 9】 答案如下

```
1      CMP     R0, R1      ; R0 与 R1 比较
2      ADDCC   R0, R0, R1   ; R0 < R1 时执行
3      SUBHI   R0, R0, R1   ; R0 > R1 时执行
```

【答案 10】 R0 = 0x0000000F

【答案 11】 ORR R0, R0, # 0x9

【答案 12】 EOR R0, R0, # 0xf

【答案 13】 答案如下

```
1      MUL     R1,R1,R6      ; R1,R1
2      MUL     R1,R6,[R5]    ; [R5]
3      MLA     R4,R5,R6      ; 少一个寄存器
4      MULSNE  R6,R3,R0      ; S 后缀与条件后缀连用时, S 放后面
5      UMLAL   R6,R4,R4,R1   ; R4,R4
6      SMULL   R5,R3,R0      ; 少一个寄存器
7      MUL     R2,R5,#0x20    ; #0x20
```

【答案 14】 答案如下

```
1    LDR R1,[R2,R5]!  
2    STR R2,[R3],#0xFFFF8    ; 超出了立即数范围  
3    STREQ R4,[R0,R4,LSL R5] ; 不能用寄存器表示移位次数  
4    STREQ R3,[R6],#-0x08  
5    LDR R0,[R2]!,-R6        ; 后索引不用! 后缀  
6    LDR R4,START  
7    LDR R1,[SP,#-0x04]  
8    LDR PC,R6                ; R6不表示一个存储地址  
9    LDR PC,[R6]  
10   LDR R1,[R3,R15]         ; R15不可作为偏移寄存器
```

【答案 15】 执行后的结果一样，区别在于第一条指令 R1 的地址偏移依赖寄存器 R4，第二条指令直接使用立即数作为固定偏移。

```
1    ; 前两条结果  
2    R0=0x00000002  
3    R1=0x00090000  
4    R4=0x00000004  
5    ; 第三条结果  
6    R0=0x00000001  
7    R1=0x00090004  
8    R4=0x00000004
```

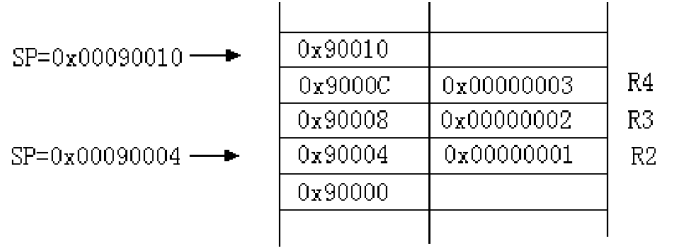
【答案 16】 结果如下

```
1      ; STMIA R4!, {R1,R2,R3}
2      mem32[0x9000]=0x00000001
3      mem32[0x9004]=0x00000002
4      mem32[0x9008]=0x00000003
5      R4=0x900C
6      ; STMDB R5!, {R1,R2,R3}
7      mem32[0x9004]=0x00000001
8      mem32[0x9008]=0x00000002
9      mem32[0x900C]=0x00000003
10     R5=0x9004
```

	0x9014
	0x9010
	0x900C
R3	0x9008
R2	0x9004
R1	0x9000

	0x9014
	0x9010
R3	0x900C
R2	0x9008
R1	0x9004
	0x9000

【答案 17】（提示：按最后一位按 0 4 8 c 0 增减）据题意：入栈时把 R2-R4 压入内存，出栈时把内存中数据存放到 R5-R7，相当于 PUSH 和 POP 操作，出栈结果是 R5=0x1、R6=0x2、R7=0x3，由于本题只问到 sp 的值，故答案只需作答两次操作后 sp 的值分别为 0x90004 和 0x90010 即可，详细过程如下：



【答案 18】 D

4 第四章 ARM 编程

4.1 ARM 伪指令

- ADR 小范围的地址读取伪指令
- ADRL 中等范围的地址读取伪指令
- LDR 大范围的地址读取伪指令
- NOP 空操作伪指令

```

1      .....
2      ADR R0,ADDR_TAB      ; 加载转换表地址
3      LDRB R1,[R0,R2]      ; 使用R2作为参数，进行查表
4      .....
5      ADDR_TAB DCB 0xA0,0xF8, 0x80,0x48, 0xE0,0x4F, 0xA3,0xD2
6      ; 通过ADR伪指令将转换表的首地址值（ADDR_TAB）加载到R0中
7      ; 每个表项的偏移地址值由R2传入，从而可以实现查表功能

```

Listing 16: ADR 伪指令加载地址

小提示：LDR 伪指令指令注意事项

- 与 ARM 指令的 LDR 相比，伪指令 LDR 的参数有“=”号
- 加载 32 位立即数或一个地址值到指定寄存器。注意“地址值”，读取时应从存储器中读取

```

1      ; LDR 加载常数
2      ; LDR 和 MOV MVN 替换
3      LDR R2, =0xFF0          ; MOV R2, #0xFF0
4      LDR R0, =0xFF000000     ; MOV R0, #0xFF000000
5      LDR R1, =0xFFFFFFF0    ; MVN R1, #0x1
6
7      ; 伪指令LDR常用于加载芯片外围功能部件的寄存器地址
8      .....
9      LDR R0,=IOPIN          ; 加载通用输入/输出GPIO的寄存器IOPIN的地址
10     LDR R1,[R0]             ; 读取IOPIN寄存器的值
11     .....
12     LDR R0,=IOSET
13     LDR R1,=0x00500500
14     STR R1,[R0]             ; IOSET = 0x00500500
15     .....

```

Listing 17: LDR 伪指令加载地址

- 常见的符号定义伪操作有如下几种
 - 用于定义全局变量的 GBLA、GBLL 和 GBLS
 - 用于定义局部变量的 LCLA、LCLL 和 LCLS
 - 用于对变量赋值的 SETA、SETL、SETS
 - 为通用寄存器列表定义名称的 RLIST
- A、L、S 含义及默认值
 - A 代表数值变量，初始化为 0
 - L 代表逻辑变量，并初始化为 F（假）
 - S 字符串变量，并初始化为空；

```

1  GBLA A1          ; 定义一个全局的数值变量，变量名为 A1
2  A1 SETA 0x0F     ; 将该变量赋值为 0x0F
3  GBLL A2          ; 定义一个全局的逻辑变量，变量名为 A2
4  A2 SETL {TRUE}   ; 将该变量赋值为真
5  GBLS A3          ; 定义一个全局的字符串变量，变量名为 A3
6  A3 SETS "Testing" ; 将该变量赋值为 "Testing"

```

Listing 18: 符号定义伪操作

数据定义伪操作

- 功能：为特定的数据分配存储单元，同时可完成已分配存储单元的初始化
- 常见操作
 - DCB 分配一片连续的字节存储单元并初始化
 - DCD (DCDU) 分配一片连续的字存储单元并初始化
 - DCDO 分配一片按字对齐的字内存单元，并初始化
 - SPACE 分配一片连续的存储单元

EQU（等价伪指令）

- 格式：名称 EQU 表达式 {, 类型}
- 功能：类似于 C 语言中的 #define
- 类型：当表达式为 32 位的常量时，可以用“类型”指示表达式的数据类型，有以下三种
 - CODE16：表明该地址处为 Thumb 指令
 - CODE32：表明该地址处为 ARM 指令
 - DATA：表明该地址处为数据区

```

1  Test EQU 50      ; 标号 Test 的值为 50
2  Addr EQU 0x55, CODE32 ; 定义 Addr 的值为 0x55，且该处为 32 位的 ARM
                        指令

```

Listing 19: 其他伪指令代码示例

全局标签

- EXPORT (全局标号声明伪指令)
- 功能: 声明一个全局的标号, 该标号可在其它的文件中引用。
- IMPORT (标号引用并加入声明伪指令)
- 功能: 声明一个外部标号名

CODE16、CODE32

- 通知编译器其后的指令按照 16 位的 Thumb 指令或 32 位的 ARM 指令编译

THUMB

- 功能: THUMB 伪指令通知编译器, 其后的指令为 Thumb 指令 (包含 Thumb-2 指令)

4.2 代码格式

```

1 ; 文件名: TEST1.S
2 ; 功能: 实现两个寄存器相加
3 ; 说明: 使用ARMulate软件仿真调试
4         AREA      Example1, CODE, READONLY      ; 声明代码段Example1
5         ENTRY                      ; 标识程序入口
6         CODE32                    ; 声明32位ARM指令
7 START   MOV  R0, #0                ; 设置参数
8         MOV  R1, #10
9 LOOP    BL   ADD_SUB               ; 调用子程序ADD_SUB
10        B     LOOP                 ; 跳转到LOOP
11 ADD_SUB
12        ADDS R0, R0, R1             ; R0 = R0 + R1
13        MOV  PC, LR                 ; 子程序返回
14        END                        ; 文件结束

```

Listing 20: ARM 代码示例

- 使用“;”进行注释
- 标号顶格写, 大小写随意
- AREA 伪操作定义一个段, 并说明所定义段的相关属性。
 - 代码段的默认属性为 READONLY
 - 数据段的默认属性为 READWRITE
 - 一个汇编程序至少应该有一个代码段
- ENTRY 伪操作标识程序的入口点, 至少一个, 可以有多个
- END 伪操作声明文件结束, 必须有一个

- 助记符可大写或小写，不能混用

```
1 X + Y 表示X与Y的和。
2 X - Y 表示X与Y的差。
3 X × Y 表示X与Y的乘积。
4 X / Y 表示X除以Y的商。
5 X: MOD: Y 表示X除以Y的余数
6 X: ROL: Y 表示将X循环左移Y位。
7 X: ROR: Y 表示将X循环右移Y位。
8 X: SHL: Y 表示将X左移Y位。
9 X: SHR: Y 表示将X右移Y位。
10 X: AND: Y 表示将X和Y按位作逻辑与操作。
11 X: OR: Y 表示将X和Y按位作逻辑或操作。
12 X: NOT: Y 表示将Y按位作逻辑非操作。
13 X: EOR: Y 表示将X和Y按位作逻辑异或操作。
```

Listing 21: 表达式和运算符

第四章例题

【例 1】判断程序及指令正误

```

1 AREA EX4_3, CODE, READONLY
2 START    ADD R0, R1, R2
3          ADD R0, R1, r2
4          add R0, R1, r2
5          Add R0, R1, r2

```

【例 2】数值表达式应用举例，下列指令汇编的结果是什么？

```

1     MOV R5, #0x20:MOD:0x04
2     ADD R4, R5, #0x20+0x20/4-8
3     MOV R4, #0xFF000000:AND:0x55000000

```

【例 3】32 位变量 X、Y 存放在存储器的地址 0x90010, 0x90014 中，要求实现 $Z = X+Y$, (Z 的值存放在 0x90018 中)

【例 4】已知 32 位有符号数 X 存放在存储器的地址 0x90010 中要求实现： $Y = \begin{cases} X & (X \geq 0) \\ -X & (X < 0) \end{cases}$
其中 Y 的值存放在 0x90010 中

【例 5】已知 32 位有符号数 X 存放在存储器的地址 0x90010 中。要求实现： $Y = \begin{cases} 1 & (X > 0) \\ 0 & (X = 0) \\ -1 & (X < 0) \end{cases}$
其中 Y 的值存放在 0x90010 中

【例 6】编制程序使 $S = 1 + 2 \times 3 + 3 \times 4 + 4 \times 5 + \dots + N(N+1)$ ，直到 N 等于 10 为止，将 S 保存在寄存器 R0 中。

【例 7】试编写一个程序，求两个数组 DATA1 和 DATA2 对应的数据之和，并把和数存入新数组 SUM 中，计算一直进行到两数之和为零结束，并把新数组的长度存入 R0 中。

```

1     AREA DATA_TEST, DATA, READWRITE
2 DATA1 DCD 2,5,0,3,-4,5,0,10,9
3 DATA2 DCD 3,5,4,-2,0,8,3,-10,5
4 SUM    DCD 0,0,0,0,0,0,0,0,0

```

【例 8】把一组数据从数据源 SRC 复制到目标数据区 DST，每次复制 1 个字，共 20 个字。要求采用单寄存器实现，编写完整程序。

```

1     AREA BlockData, DATA, READWRITE
2 SRC    DCD 1,2,3,4,5,6,7,8,1,2,3,4,5,6,7,8,1,2,3,4
3 DST    DCD 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0

```

【例 9】把一组数据从数据源 SRC 复制到目标数据区 DST

- 复制时以 8 个字为单位进行
- 对于最后所剩不足 8 个字的按字进行赋值
- 在进行以 8 个字为单位的数据复制时，保存了所用的 8 个工作寄存器。
- 数据定义同上题

【例 10】在以 BUF 为首址的字存储区中存放有 10 个无符号数 0x0FF,0x00,0x40,0x10,0x90,0x20,0x80,0x30,0x50,0x70,0x60，现需将它们按从小到大的顺序排列在 BUF 存储区中，编写其程序。(冒泡排序)

【例 11】完成两个 128 位二进制数的加法，第一个数由高到低存放在寄存器 R7~R4，第二个数由高到低存放在寄存器 R11~R8，运算结果由高到低存放在寄存器 R3~R0

第四章答案

- 【答案 1】
- 没有程序入口
 - 没有 END
 - 第一行前面没有空格
 - 第五行助记符大小写混用

【答案 2】 以上指令汇编后分别等价于如下指令

```

1      MOV R5,#0
2      ADD R4,R5,#0x20
3      MOV R4,#0x55000000

```

【答案 3】 答案如下

```

1      AREA EX4_3, CODE, READONLY
2      ENTRY
3      CODE32
4  START    LDR R0,=0x90010      ; 变量X的地址送入R0
5           LDR R1,[R0],#4      ; 变量X的值读入R1
6           LDR R2,[R0],#4      ; 变量Y的值读入R2
7           ADD R1,R1,R2        ; X+Y结果存入R1
8           STR R1,[R0]         ; 结果存入Z中
9           B START             ; 死循环
10          END

```

【答案 4】 分析：该程序的结构很简单，首先将位于地址 0x90010 处的数值取出来和 0 比较，如果小于 0 则将其取负，然后存回原地址处。这里满足条件将其取负只需通过 SUBLT R2,R0,R2 来实现，因而可以通过条件标志 LT 来控制该指令执行与否，从而实现一个简单的分支。

```

1      AREA E4_4, CODE, READONLY
2      ENTRY
3      CODE32
4  START    LDR R1,=0x90010
5           MOV R0,#0
6           LDR R2,[R1]
7           CMP R2,#0
8           SUBLT R2,R0,R2
9           STR R2,[R1]
10          END

```

【答案 5】 答案不唯一

```

1      AREA E4_5, CODE, READONLY
2      ENTRY

```

```

3          CODE32
4 START    LDR R1,=0X90010
5          LDR R2,[R1]
6          CMP R2,#0
7          BEQ ZERO
8          BGT PLUS
9          MOV R0,#-1
10         B FINISH
11 PLUS    MOV R0,#1
12         B FINISH
13 ZERO    MOV R0,#0
14 FINISH  STR R0,[R1]
15         END

```

【答案 6】 答案不唯一

```

1      AREA E4_6, CODE, READONLY
2      ENTRY
3      CODE32
4 START
5      MOV R0,#1
6      MOV R1,#2
7 REPEAT
8      ADD R2,R1,#1
9      MUL R3,R2,R1
10     ADD R0,R0,R3
11     ADD R1,R1,#1
12     CMP R1,#10
13     BLE REPEAT
14     END

```

【答案 7】 答案不唯一

```

1      AREA E4_7, CODE, READONLY
2      ENTRY
3      CODE32
4
5 start
6      LDR R1, =DATA1      ; R1 指向数组 DATA1 的首地址
7      LDR R2, =DATA2      ; R2 指向数组 DATA2 的首地址
8      LDR R3, =SUM        ; R3 指向数组 SUM 的首地址
9      MOV R4, #0          ; R4 作为数组 SUM 的长度计数器
10
11 loop

```

```

12     LDR R5, [R1], #4    ; 从 DATA1 中读取一个值到R5, 并更新R1指针
13     LDR R6, [R2], #4    ; 从 DATA2 中读取一个值到R6, 并更新R2指针
14     ADD R7, R5, R6      ; 计算 R5 和 R6 的和, 并存入 R7
15     STR R7, [R3], #4    ; 将和存入 SUM, 并更新 R3 指针
16     ADD R4, R4, #1      ; SUM 的长度计数器加 1
17     CMP R7, #0          ; 比较 R7 和 0
18     BNE loop            ; 如果和不为零, 继续循环
19
20 end
21     MOV R0, R4           ; 将 SUM 的长度存入 R0
22     B end                ; 无限循环, 停止程序
23
24     AREA DATA_TEST, DATA, READWRITE
25 DATA1 DCD 2,5,0,3,-4,5,0,10,9
26 DATA2 DCD 3,5,4,-2,0,8,3,-10,5
27 SUM DCD 0,0,0,0,0,0,0,0,0,0
28     END

```

【答案 8】 答案不唯一

```

1     AREA E4_8, CODE, READONLY
2 num EQU 20 ; 设置复制的字数
3     ENTRY ; 程序入口
4     CODE32
5 start LDR r0, =SRC ; r0指向源数据块首地址
6       LDR r1, =DST ; r1指向目标数据块首地址
7       MOV r2, #num ; r2复制的字数
8 wordcopy
9       LDR r3, [r0], #4 ; 从数据源取出一个字数据到R3, 并更新R0
10      STR r3, [r1], #4 ; 将字数据存储到目标地址, 并更新R1
11      SUBS r2, r2, #1 ; 字复制次数减1
12      BNE wordcopy ; 循环, 直到字复制完毕
13      AREA BlockData, DATA, READWRITE
14 SRC DCD 1,2,3,4,5,6,7,8,1,2,3,4,5,6,7,8,1,2,3,4
15 DST DCD 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
16     END

```

【答案 9】 答案不唯一

```

1     AREA E4_9, CODE, READONLY
2     ENTRY
3
4     LDR R0, =SRC ; R0 指向 SRC
5     LDR R1, =DST ; R1 指向 DST

```

```

6      MOV R2, #20          ; R2 初始化为 20, 表示要复制的总字
      数
7      MOV SP, #0x400      ; 设置堆栈指针用于保存工作寄存器
8
9  COPY_BLOCKS
10     MOV R3, R2, LSR #3   ; 计算需要复制的 8 字块数
11     BEQ COPY_REMAINDER  ; 如果不足一个 8 字块, 跳转到
      COPY_REMAINDER
12     STMFD SP!, {R4-R11} ; 保存工作寄存器
13
14  COPY_8WORDS
15     LDMIA R0!, {R4-R11} ; 从 SRC 读取 8 个字到寄存器组, 并
      更新 R0
16     STMIA R1!, {R4-R11} ; 将寄存器组内容写入 DST, 并更新 R1
17     SUBS R3, R3, #1      ; 块计数器减 1
18     BNE COPY_8WORDS     ; 如果块计数器不为 0, 继续复制
19
20     LDMFD SP!, {R4-R11} ; 恢复工作寄存器
21
22  COPY_REMAINDER
23     ANDS R2, R2, #7      ; 计算剩余不足 8 个字的数量
24     BEQ END_COPY        ; 如果没有剩余, 跳转到 END_COPY
25
26  COPY_WORDS
27     LDR R3, [R0], #4     ; 从 SRC 读取一个字, 并更新 R0
28     STR R3, [R1], #4     ; 将一个字写入 DST, 并更新 R1
29     SUBS R2, R2, #1      ; 剩余字计数器减 1
30     BNE COPY_WORDS      ; 如果剩余字计数器不为 0, 继续复制
31
32  END_COPY
33     B END_COPY          ; 程序结束的无限循环
34
35     AREA BlockData, DATA, READWRITE
36  SRC     DCD 1,2,3,4,5,6,7,8,1,2,3,4,5,6,7,8,1,2,3,4
37  DST     DCD 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
38      END

```

【答案 10】 答案不唯一, 先设计需要使用的寄存器, 以及进行数据区定义

```

1  #include <stdio.h>
2  #define N 10
3
4  int main() {

```

```

5      int BUF[N] = {0xFF, 0x00, 0x40, 0x10, 0x90, 0x20, 0x80, 0
6          x30, 0x50, 0x70};
7
8      for (int i = 0; i < N - 1; i++) {
9          for (int j = 0; j < N - 1 - i; j++) {
10             if (BUF[j] > BUF[j + 1]) {
11                 int temp = BUF[j];
12                 BUF[j] = BUF[j + 1];
13                 BUF[j + 1] = temp;
14             }
15         }
16     }
17
18     for (int i = 0; i < N; i++) {
19         printf("0x%X ", BUF[i]);
20     }
21
22     return 0;
23 }

```

```

1      AREA BubbleSort, CODE, READONLY
2      ENTRY
3
4      N      EQU 10
5
6      CODE32
7      LDR R0, =BUF      ; R0指向数组的首地址
8      MOV R1, #0        ; 外循环计数器 i
9
10     OUTER_LOOP
11         CMP R1, #N-1    ; 检查是否外循环结束 (i < N-1)
12         BGE END_SORT    ; 如果 i >= N-1, 跳转到结束
13
14         MOV R2, #0      ; 内循环计数器 j
15
16     INNER_LOOP
17         MOV R3, #N-1    ; 检查是否内循环结束 (j < N-1-i)
18         SUB R3, R3, R1    ; 计算 N-1-i
19         CMP R2, R3
20         BGE INCREMENT_OUTER ; 如果 j >= N-1-i, 跳转到外循环增量
21
22         ADD R4, R0, R2, LSL #2 ; R4 = BUF + j
23         LDR R5, [R4]      ; R5 = BUF[j]

```

```

24      LDR R6, [R4, #4]          ; R6 = BUF[j+1]
25      CMP R5, R6                ; 比较 BUF[j] 和 BUF[j+1]
26      BLE INCREMENT_INNER      ; 如果 BUF[j] <= BUF[j+1], 跳转
                                ; 到内循环增量
27
28      ; 交换 BUF[j] 和 BUF[j+1]
29      STR R6, [R4]              ; BUF[j] = R6
30      STR R5, [R4, #4]          ; BUF[j+1] = R5
31
32 INCREMENT_INNER
33      ADD R2, R2, #1            ; j++
34      B INNER_LOOP              ; 跳回内循环
35
36 INCREMENT_OUTER
37      ADD R1, R1, #1            ; i++
38      B OUTER_LOOP              ; 跳回外循环
39
40 END_SORT
41      ; 排序结束, 程序可以在这里停止或继续其他操作
42      B END_SORT                ; 无限循环, 程序结束
43
44      AREA BlockData, DATA, READWRITE
45 BUF    DCD 0xFF,0x00,0x40,0x10,0x90,0x20,0x80,0x30,0x50,0x70
46      END

```

【答案 11】 答案如下

```

1      ADDS R0, R4, R8           ; 加低端的字
2      ADCS R1, R5, R9           ; 加第二个字, 带进位
3      ADCS R2, R6, R10          ; 加第三个字, 带进位
4      ADC  R3, R7, R11          ; 加第四个字, 带进位

```

5 第五章 C 语言与汇编语言混编

5.1 概念

1. 参数传递规则

- 当参数 ≤ 4 个时，使用寄存器 R0~R3 来传递参数
- 当参数 > 4 个时，使用数据栈来传递参数
- 入栈的顺序与参数顺序相反，最后一个字数据先入栈

2. 嵌入式 C 语言与 C 语言编程的不同（6 点）

- 数据类型尽量考虑整型；
- 计算符号多使用加减，少使用乘法，不使用除法。尽量采用移位运算代替乘法和除法；
- 变量尽量用局部变量，少使用全局变量；
- 子程序嵌套越少越好；
- 循环体多用 do-while，少用 for、while 等结构；
- 子程序参数个数受限制（根据混合编程规范，不多于四个参数）。

3. while 循环与 ARM 汇编代码比较

```

1 while(x < 10){
2     x = x + 1;
3 }

```

```

1 B      chack
2 loop  ADDS r1, r1, #1
3 chack CMP  r1, #0xa
4      BCC  loop

```

4. for 循环与 ARM 汇编代码比较

```

1 for (i = 0; i < 10; i++){
2     x += i;
3 }

```

```

1      MOVS    r3, #0
2      B      chack
3 loop  ADDS    r1, r1, r3
4      ADDS    r3, r3, #1
5 chack CMP     r3, #0xa
6      BCC     loop

```

5. do-while 循环与 ARM 汇编代码比较

```

1 do{
2     x += 2;
3 }while(x < 20);

```

```

1 loop  ADDS r1, r1, #2
2      CMP  r1, #0x14
3      BCC  loop

```


5.2 ARM 和 Thumb 混合编程

- 从 ARM5 版本开始，实现程序状态的切换的指令有：
 - BX、BLX
 - LDR、LDM 及 POP
- BX：带状态切换的分支指令
 - 格式：BX Rm
 - 功能：BX 指令跳转到指令中所指定的目标地址，并实现状态的切换
 - 操作：PC←Rm，切换处理器状态
 - 解释：指令可以根据跳转地址（Rm）的最低位来切换处理器状态。其跳转范围限制在当前指令的 $\pm 32\text{M}$ 字节地址内（ARM 指令为字对齐，最低 2 位地址固定为 0）
 - Rm 的位 [0] 不用作地址的一部分
 - 若 Rm 的位 [0] 为 1，则指令将 CPSR 中的标志 T 置位，且将目标地址的代码解释为 Thumb 代码
 - 若 Rm 的位 [0] 为 0，将目标地址的代码解释为 ARM 代码，且 Rm 的位 [1] 就不能为 1

* 探究：BX 如何切换及地址寻址

- 假设 Rm 的值为 0x20000001：
bit[0] 为 1，表示跳转到 Thumb 状态。此时地址为 0x20000000（忽略 bit[0]），满足 Thumb 指令集的 2 字节对齐要求。
- 假设 Rm 的值为 0x20000000：
bit[0] 为 0，表示跳转到 ARM 状态。此时地址为 0x20000000（忽略 bit[0]），满足 ARM 指令集的 4 字节对齐要求。
- 假设 Rm 的值为 0x20000002：
bit[0] 为 0，表示跳转到 ARM 状态。但 bit[1] 为 1，此地址不能满足 ARM 指令集的 4 字节对齐要求，所以这是无效的。
- 目的寄存器 Rd（包括 Rdlo、Rdhi）和操作数 Rm 必须是不同的寄存器

```

1      CODE32                ; ARM 程序段，32位编码
2 arm1  ADR R0,thumb1+1      ; 伪指令，把标号thumb1所在的地址赋给R0，末
    位R0[0]置1，要跳转到THUMB指令集
3      .....
4      MOV LR,PC            ; 设置返回地址
5      BX  R0                ; 跳转
6      ADD R1,R2,#2          ; 返回地址处，第4条指令
7
8      CODE16                ; THUMB程序段，16位编码
9 thumb1 ADD R1,R3,#1        ; THUMB程序
  
```

```
10      .....
11      BX LR                ; 跳转到返回地址处，执行第4条指令
```

Listing 22: BX 指令实现状态切换例子

- BLX：带返回和状态切换的分支指令
 - 格式：BLX Rm
 - 操作：LR ← 返回地址，PC ← Rm，切换处理器状态
 - 注意：对于 cortex-M3 系统，Rm 的 bit[0] 必须是 1，不能进入 ARM 状态，否则系统会出现异常
 - 区别：BLX 指令连接寄存器 LR 中自动设置了返回地址，不需要使用 MOV LR, PC 指令来保存 PC 的值，简化了 Thumb 例程的调用

```
1      CODE32                ; ARM程序段，32位编码
2  arm1      ADR R0,thumb1+1    ; 把标号thumb1所在的地址赋给R0，将R0的
      bit[0]置1
3      .....
4      ; 不需要 MOV LR,PC 设置返回地址
5      BLX R0                ; 跳转，同时设置返回地址 即 PC ← LR
6      ADD R1,R2,#2          ; 返回地址处，第4条指令
7
8      CODE16                ; THUMB程序段，16位编码
9  thumb1    ADD R1,R3,#1      ; THUMB程序
10     .....
11     BX LR                ; 跳转到返回地址处，执行第4条指令
```

Listing 23: BLX 指令实现状态切换例子

```
1      AREA EX4_53, CODE, READONLY
2      ENTRY
3      CODE32
4  MAIN      ADR R0, THUMBCODE+1 ;将R0的bit[0]置1
5      BX R0                ;跳转，并根据R0的bit[0]实现状态切换
6
7      CODE16
8  THUMBCODE MOV R2, #2
9      MOV R3, #3
10     ADD R2, R2, R3
11     ADR R0, ARMCODE        ;加载ARMCODE地址到R0
12     BX R0
13
14     CODE32
15  ARMCODE   MOV R4, #4
```

```
16      MOV    R5, #5
17      ADD    R4, R4, R5
18      B      MAIN
19                                     END
```

Listing 24: BX 指令实现状态切换

```
1      CODE32
2 arm    LDR    R0, =thumb1 ; 标号 thumb1 的地址赋给 R0
3        LDR    R1, =arm1   ; 标号 arm1 的地址赋给 R1
4        STMFD  SP!,{R1,R0} ; R1,R0 压栈, SP 指向 R0
5        .....             ; (ARM 程序)
6        LDMFD  SP!,{PC}    ; R0 出栈, 即 PC ← thumb1, 跳至 thumb 状态
7      CODE16
8 thumb1 POP    {PC}        ; R1 出栈, 即 PC ← arm1, 返回 arm 状态
9                                     END
```

Listing 25: LDR、LDM、POP 指令实现状态切换的例子

5.3 汇编语言和 C 语言交互编程

- 汇编程序访问 C 程序变量
 1. 在 C 程序中：声明的全局变量
 2. 汇编程序：
 - 使用 IMPORT/EXTERN 伪指令声明/引用该全局变量
 - 利用 LDR 伪指令读取该全局变量的内存地址
 - 对于该数据的类型，采用不同选项的 LDR 指令读取该全局变量，使用相应的 STR 指令存储该全局变量。
- 汇编程序调用 C 程序
 1. 在 C 程序中：不需要进行操作
 2. 汇编程序：需要使用 IMPORT 声明 C 程序、注意参数传递、通过 BL 指令来调用 C 程序
- C 程序调用汇编程序
 1. 在 C 程序中：用 EXTERN 关键字声明需要调用的汇编函数原型
 2. 汇编程序：
 - 用上述声明的函数名作为汇编代码段的标识
 - 使用 EXPORT 声明上述函数可被外部调用
 - 函数代码的最后用 MOV PC,LR 返回

C 变量类型	访问的汇编指令
unsigned char	LDRB/STRB
unsigned short	LDRH/STRH
unsigned int	LDR/STR
char	LDRSB/STRSB
short	LDRSH/STRSH

表 5: 访问 C 程序变量的对应汇编指令

```
1 // C语言源程序str.c
2 #include<stdio.h>
3 int globvar=3;  //定义一个整型全局变量（1）
4 int main()
5 {
6     return 0;
7 }

; 汇编文件hello.s
AREA EX4_52,CODE,READONLY
EXPORT ARMCODE ; 用EXPORT伪操作声明该变量可被外部引用
IMPORT globvar ; 用IMPORT伪操作声明该变量是外部定义的
ENTRY
ARMCODE LDR R1,=globvar ; 将其地址读入R1（3）
        LDR R0,[R1] ; 将其值读入R0（4）
        ADD R0,R0,#2 ; 修改R0的值
        STR R0,[R1] ; 将R0的值赋予变量，修改变量的值（5
        )
        MOV PC,LR
END
```

Listing 26: 汇编程序访问 C 程序变量例子

- C 程序实现 5 个整数求和

```

1 // C语言源程序C_add.c
2 #include<stdio.h>
3 int g(int a,int b,int c,int d,int e)
4 {
5     return a+b+c+d+e;
6 }

```

```

1 ; 汇编语言源程序ARM_add.s
2     AREA ARM_add,CODE,READONLY
3     EXPORT callsum ; 使用EXPORT声明可被外部程序引用
4     IMPORT g       ; 使用IMPORT伪操作声明C程序g( )
5     ENTRY
6     LDR SP,=0xA000
7 callsum STR LR,[SP,#-4]!    ; 保存返回地址
8     MOV R0,#1            ; 设置参数1
9     MOV R1,#2            ; 设置参数2
10    MOV R2,#3            ; 设置参数3
11    MOV R3,#4            ; 设置参数4
12    MOV R4,#5            ; 参数5通过数据栈传递
13    STR R4,[SP,#-4]!
14    BL g                  ; 调用C程序g( ), 其结果从R0返回
15    ADD SP,SP,#4          ; 调整数据栈指针, 准备返回
16    LDR PC,[SP],#4        ; 返回
17    END

```

Listing 27: 汇编程序调用 C 程序例子

- 第 6 行: LDR SP,=0xA000 使得 SP 被初始化为 0xA000, 为后续的栈操作提供一个已知的起点, 从而确保程序执行过程中栈的管理是正确且可控的。
- 第 7 行: LR (Link Register) 是用于存储函数调用返回地址的寄存器。当使用 BL (Branch with Link) 指令调用另一个函数时, CPU 会自动将下一条指令的地址保存到 LR 中。这样, 当被调用的函数执行完毕后, 可以使用 LR 中的地址返回到调用点继续执行。因此, 如果其他函数调用 callsum 函数 (假设为函数 haha), 此时 LR 中存的是 haha 的返回值。由于本函数会调用 g 函数, 则 LR 的值会被更新成第 15 行的地址, 那么执行完本函数就没法返回 haha 函数了, 因此需要把 haha 的函数返回值保存下来, 类似于之前讲堆栈时说的为了解决寄存器不够用的情况。如果不涉及复杂的函数调用链, 可以去掉这句话, 更改第 16 行语句为 B 语句也可。
- 第 8-13 行: 给 g 函数传递参数; ARM 架构规定: 前四个参数通过 R0-R3 寄存器传递。超过四个参数时, 通过栈传递。
- 第 14 行: 调用 C 函数 g, 并将返回值 (下一条指令地址) 存入 LR 中

- 第 15 行：移除之前存储的第五个参数。
- 第 16 行：从栈中恢复 LR 的值到程序计数器 (PC) 中，这样程序会返回到调用 `callsum` 的地方

C 语言程序调用汇编程序例子

- 汇编程序 `strcpy` 实现字符串的复制功能；
- C 程序调用 `strcpy` 完成字符串复制工作。

```

1 //  strtest.c 源程序
2 #include <stdio.h>
3
4 //使用 关键词EXTERN 声明 strcpy
5 extern void strcpy(char *d, const char *s);
6
7 int main()
8 {
9     const char *srcstr = "First string - source";
10    char dststr[] = "Second string - destination";
11    /* dststr is an array since we're going to change it */
12
13    printf("Before copying:\n");
14    printf("  '%s'\n  '%s'\n",srcstr,dststr);
15    strcpy(dststr,srcstr);    //将源串和目标串的地址传递给strcpy
16    printf("After copying:\n");
17    printf("  '%s'\n  '%s'\n",srcstr,dststr);
18    return 0;
19 }

```

```

1 ; 汇编语言源程序 ARM_add.s
2     AREA SCopy, CODE, READONLY
3     EXPORT  strcpy           ; 使用EXPORT声明可被外部程序引用
4
5 strcpy                ; r0 指向目标地址, r1 指向源地址
6     LDRB r2, [r1], #1    ; 从源地址加载字节数据并修改地址指针
7     STRB r2, [r0], #1    ; 存储字节到目标地址并修改地址指针;
8     CMP  r2, #0          ; 遇到0则结束
9     BNE  strcpy          ; 否则继续复制
10    MOV  pc, lr           ; 程序返回
11    END

```

Listing 28: C 语言程序调用汇编程序例子

第五章例题

【例 1】编写两数（1，5）相加的汇编语言函数，用 C 程序调用这个汇编函数

【例 2】编写两数（1，5）相加的 C 语言函数，用汇编程序调用这个 C 函数

【例 3】找出三个数中最小的数并返回（利用汇编程序调用 C 程序）

第五章答案

【答案 1】（答案不唯一）提示：C 程序: funcsum(1, 5) 调用会将 1 传递到寄存器 R0, 5 传递到寄存器 R1；不要忘记汇编返回 C 函数

```

1 // test_add.c
2 #include <stdio.h>
3
4 extern int funcsum(int arg1, int arg2);
5
6 int main() {
7     int product = funcsum(1, 5);
8     printf("%d\n", product);
9     return 0;
10 }
```

```

1     AREA Block, CODE, READONLY
2     EXPORT funcsum
3     ENTRY
4 funcsum
5     ADD R0, R0, R1
6     MOV PC, LR
7     END
```

【答案 2】（答案不唯一）sum 函数执行完毕后，由第八条指令返回调用 BL sum 的地方继续执行。

```

1 // test_add.c
2 #include <stdio.h>
3 int sum(int a, int b)
4 {
5     return a+b;
6 }
```

```

1     AREA Block, CODE,
2         READONLY
3     IMPORT sum
4     ENTRY
5 funcsum
6     MOV R0, #1
7     MOV R1, #5
8     BL sum
9     MOV PC, LR
10    END
```

【答案 3】 答案不唯一

```

1 #include <stdio.h>
2 int findMin(int a, int b, int
   c) {
3     if (a < b) {
4         if (a < c) {
5             return a;
6         }
7         else {
8             return c;
9         }
10    }
11    else {
12        if (b < c) {
13            return b;
14        }
15        else {
16            return c;
17        }
18    }
19 }

```

```

1     AREA Block, CODE,
      READONLY
2     IMPORT findMin
3     ENTRY
4 funcsum
5     MOV R0, #a
6     MOV R1, #b
7     MOV R2, #c
8     BL findMin
9     MOV PC, LR
10    END

```


6 第六章 STM32 输入输出

6.1 基本概念

6.1.1 GPIO 的 8 种工作模式

四种输入模式：模拟输入、浮空输入、上拉输入、下拉输入

四种输出模式：开漏输出、开漏复用输出、推挽式输出、推挽复用输出

6.1.2 输入／输出常用固件库函数

初始化外设 GPIOD 寄存器；

- GPIO_Init

2 个设置输出电平函数

- GPIO_SetBits：设置某个 IO 口输出为高电平 (1)
- GPIO_ResetBits：设置某个 IO 口输出为低电平 (0)

6.2 编程

```

1 typedef struct
2 {
3     uint16_t GPIO_Pin;           // 指定要初始化的IO口
4     GPIOSpeed_TypeDef GPIO_Speed; // 设置IO口输出速度
5     GPIOMode_TypeDef GPIO_Mode;  // 设置工作模式：8种中的一个
6 }GPIO_InitTypeDef;
```

```

1 GPIO_InitTypeDef GPIO_InitStructure;
2 GPIO_InitStructure.GPIO_Pin      = GPIO_Pin_All;
3 GPIO_InitStructure.GPIO_Speed   = GPIO_Speed_50MHz;
4 GPIO_InitStructure.GPIO_Mode    = GPIO_Mode_Out_PP;
5     //IO配置为推挽输出口
6
7 GPIO_Init(GPIOA, &GPIO_InitStructure);
```

Listing 29: 初始化 GPIO 端口操作

```

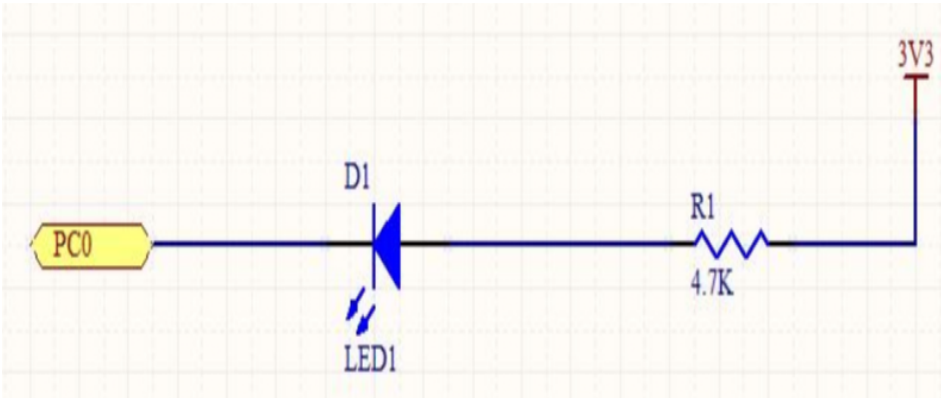
1 struct
2 {
3     char *name;
4     int age;
5     int grade;
6 }Stu;
```

```

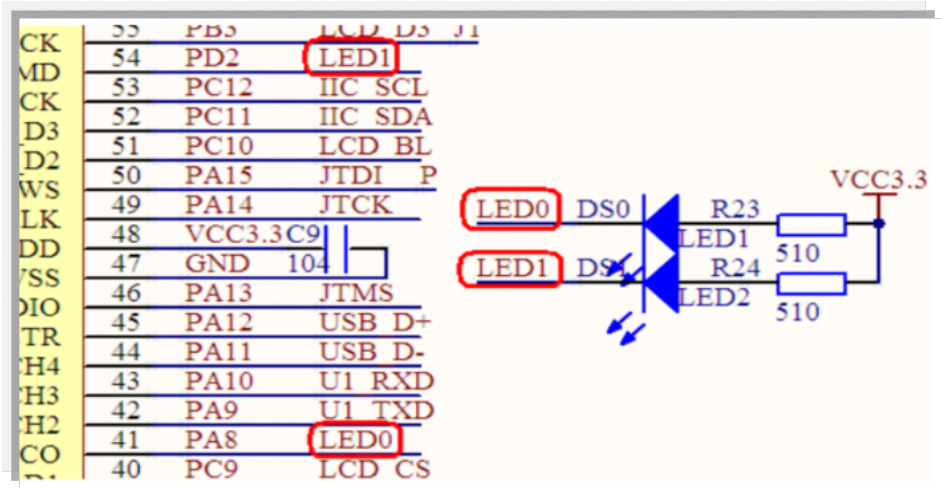
1 Stu lxc;
2 lxc.name = "林星辰";
3 lxc.age  = 21;
4 lxc.grade = 3;
```

第六章例题

【例 1】 请根据以下电路图点亮 LED 小灯



【例 2】 根据以下电路图实现 LED 跑马灯



第六章答案

【答案 1】 答案如下

```
1 #include "stm32f10x.h"
2 int main(void)
3 {
4     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC,ENABLE);
5                                     //使能GPIOC的时钟
6
7     GPIO_InitTypeDef  GPIO_InitStructure;
8     GPIO_InitStructure.GPIO_Pin=GPIO_Pin_0;
9     GPIO_InitStructure.GPIO_Speed=GPIO_Speed_50MHz;
10    GPIO_InitStructure.GPIO_Mode=GPIO_Mode_Out_PP;
11    GPIO_Init(GPIOC,&GPIO_InitStructure);
```

```
12     GPIO_ResetBits(GPIOC,GPIO_Pin_0);    // 阴极点亮led灯
13 }
```

【答案 2】 答案如下

```
1  #include "stm32f10x.h"
2  #include "delay.h"
3
4  int main(void)
5  {
6      GPIO_InitTypeDef  GPIO_InitStructure;
7      delay_init();
8      // GPIOD 时钟使能
9      RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOD, ENABLE);
10     // GPIOA 时钟使能
11     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);
12
13     // 配置GPIOD 引脚
14     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
15     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_ALL;
16     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
17
18     GPIO_Init(GPIOD, &GPIO_InitStructure);
19     GPIO_Init(GPIOA, &GPIO_InitStructure);
20
21     while(1)
22     {
23         GPIO_SetBits(GPIOD,GPIO_Pin_2); //PD.2 输出高电平
24         GPIO_ResetBits(GPIOD,GPIO_Pin_2); //PD.2 输出低电平
25
26         delay_ms(500);
27         GPIO_SetBits(GPIOA,GPIO_Pin_8); //PA.8 输出高电平
28         GPIO_ResetBits(GPIOA,GPIO_Pin_8); //PA.8 输出低电平
29         delay_ms(500);
30     }
31
32 }
```